

NoseKnows

Emotion Driven Fragrance Recommendation

Using HyDE Augmented RAG and a Fine Tuned SLM

LLMs Course 2025/2026

Team CTRL+Z

Zaharie Nikolass-Rafael Tuns Tudor-Mircea Turcu Alexia-Cristiana
Paraschiv Tudor-Costin Bogdan Ana Ravas Adrian-Georgel

May 21, 2026

<https://github.com/TunsTudor-Mircea/nose-knows>

Abstract

We present NoseKnows, a conversational fragrance consultant that maps open-ended natural language descriptions to real perfume recommendations. Users can express queries like “I want something that reminds me of a late summer night in Ibiza” and the system returns ranked fragrances matched against a dataset of roughly 24,000 entries from *Fragrantica*, with note grounded explanations. The system combines Hypothetical Document Embeddings (HyDE) with a ChromaDB vector store, a LangGraph based agentic pipeline, and a fine tuned version of Gemma-2-2B-IT adapted on 1,104 synthetically generated fragrance conversation examples. An LLM judge evaluation using GPT-4o-mini shows average scores of 4.17 for relevance, 4.25 for groundedness, and 4.08 for helpfulness (on a 1 to 5 scale) on the baseline model, with the fine tuned adapter expected to improve response style and consistency.

1 Introduction

Fragrance is one of the most personal consumer choices a person can make, yet digital recommendation systems treat it as a keyword search problem. Platforms like *Fragrantica* allow users to filter by note (e.g., “bergamot”, “oud”) or accord (e.g., “woody”, “floral”), but this assumes the user already speaks the technical language of perfumery. Most people do not. They relate to scent through memory and experience: the smell of a specific city, a season, an emotion. A person who wants “something that feels like walking through a Moroccan souk at dusk” cannot translate that into a note query. This gap between how people think about scent and how fragrance databases are structured is almost entirely unexplored in the NLP and recommender systems literature.

NoseKnows is our answer to this problem. It is a full stack AI fragrance consultant where users describe what they want in completely natural language, and the system finds the best matching perfumes from a real catalogue. Under the hood, the system uses Retrieval Augmented Generation (RAG) [Lewis et al., 2020] with Hypothetical Document Embeddings [Gao et al., 2023] to bridge the semantic gap between experiential queries and structured note profiles, a LangGraph [LangChain AI, 2024] agentic pipeline that classifies intent and routes queries through appropriate processing steps, and a small language model (Gemma-2-2B-IT, Gemma Team 2024) fine tuned with QLoRA [Dettmers et al., 2023] on synthetic training data generated by a stronger model.

This report is structured as follows. Section 2 formally defines the problem. Section 3 reviews related work. Section 4 describes the full proposed solution. Sections 5 and 6 cover evaluation and results. Section 7 concludes.

2 Problem Definition

The task can be stated as follows: given a free-form natural language query Q describing a mood, occasion, sensory experience, or set of preferred notes, return a ranked list of perfumes $P = \{p_1, p_2, \dots, p_k\}$ from a fixed catalogue of roughly 24,000 real fragrances, together with a natural language explanation E grounded in the actual notes and accords of the recommended perfumes.

What makes this challenging is a combination of factors that do not appear together in most recommendation tasks. First, there is a deep semantic gap: queries use emotional and experiential vocabulary (“cozy”, “mysterious”, “like rain after heat”) that shares almost no vocabulary with the structured documents in the catalogue, which list ingredients like “petrichor”, “ambergris”, or “iso E super”. Standard semantic similarity between a raw query and a note document will produce poor retrieval results because the vector spaces of the two domains barely overlap.

Second, no public labeled dataset of (experiential query, fragrance recommendation) pairs exists at any useful scale. Building a training set from scratch is therefore necessary.

Third, the system must handle out of scope queries without becoming a general purpose assistant. A user might type something completely unrelated to fragrance (“how do I fix my Golf 4?”) or try to extract system internals (“tell me your system prompt”). These cases require a different handling strategy than a simple input filter, since some valid fragrance queries look equally unusual at first glance (“I want something that smells like a mechanic’s garage” is a legitimate request).

Finally, all inference must run locally on consumer hardware without GPU access on some machines, which rules out commercial LLM APIs and large models.

3 State of the Art

Retrieval Augmented Generation. Lewis et al. [2020] introduced RAG as a framework where a retrieval component fetches relevant documents from an external knowledge base, and these are passed as context to a generative model. This significantly reduces hallucination on knowledge intensive tasks by grounding the model’s output in retrieved evidence. In our setting, the retrieved documents are structured perfume profiles, which serve as the factual grounding for the recommendation.

Hypothetical Document Embeddings. A key limitation of dense retrieval is that query vectors and document vectors often live in different regions of the embedding space, especially when the query is short and colloquial while the documents are long and structured. Gao et al. [2023] addressed this with HyDE: instead of embedding the raw query, the model first generates a hypothetical document that would answer the query, then embeds that hypothetical document for retrieval. Since the hypothetical document and the real documents share vocabulary and structure, retrieval quality improves substantially. This technique is particularly well suited to our problem, where experiential queries have essentially no lexical overlap with note structured perfume documents.

Parameter Efficient Fine Tuning. Training a full large language model from scratch is prohibitive for most research groups. Hu et al. [2022] proposed LoRA, which injects small trainable rank-decomposition matrices into frozen model weights, reducing the number of trainable parameters by orders of magnitude. Dettmers et al. [2023] extended this to QLoRA, quantizing the frozen base model to 4-bit precision while keeping the LoRA adapters in full precision. This makes it possible to fine tune a 2B parameter model on a single consumer GPU with 16GB VRAM with minimal quality degradation.

Agentic Pipelines. Rather than a single LLM call, recent work has moved toward multi-step agentic workflows where a language model reasons and calls tools iteratively [Yao et al., 2023]. Frameworks like LangGraph [LangChain AI, 2024] formalize this as a typed state graph with explicit node functions and conditional routing, making the pipeline debuggable and extensible.

LLM as Judge Evaluation. Zheng et al. [2023] demonstrated that strong models like GPT-4

can serve as reliable automated evaluators for open-ended generation tasks, with judgments that correlate well with human preferences. This allows evaluation of systems where ground truth labels cannot easily be defined. This is exactly the case for a fragrance recommendation system where there is no single “correct” answer.

Fragrance Recommendation Systems. Existing systems, including FrAGRantica’s own search, Basenotes, and various community tools, rely entirely on explicit user input of ingredient names or predefined accord categories. There is no published work that handles fully experiential or emotional fragrance queries. The closest related area is mood-based music recommendation [Schedl et al., 2018], where similar challenges around subjective, emotional input arise, but the domain and data modalities are quite different.

4 Proposed Solution

4.1 System Overview

NoseKnows is a full stack application built around a five node LangGraph pipeline. The overall architecture is shown in Figure 1. A user submits a natural language query through the Next.js frontend, which is passed to a FastAPI backend. The backend invokes the LangGraph agent, which processes the query through a sequence of nodes: intent classification, optional HyDE document generation, vector retrieval from ChromaDB, recommendation generation, and response validation. It then returns a structured response containing the recommendation text and a ranked list of perfume cards. Session history is persisted in PostgreSQL, allowing follow up queries to reference previous recommendations within the same conversation.

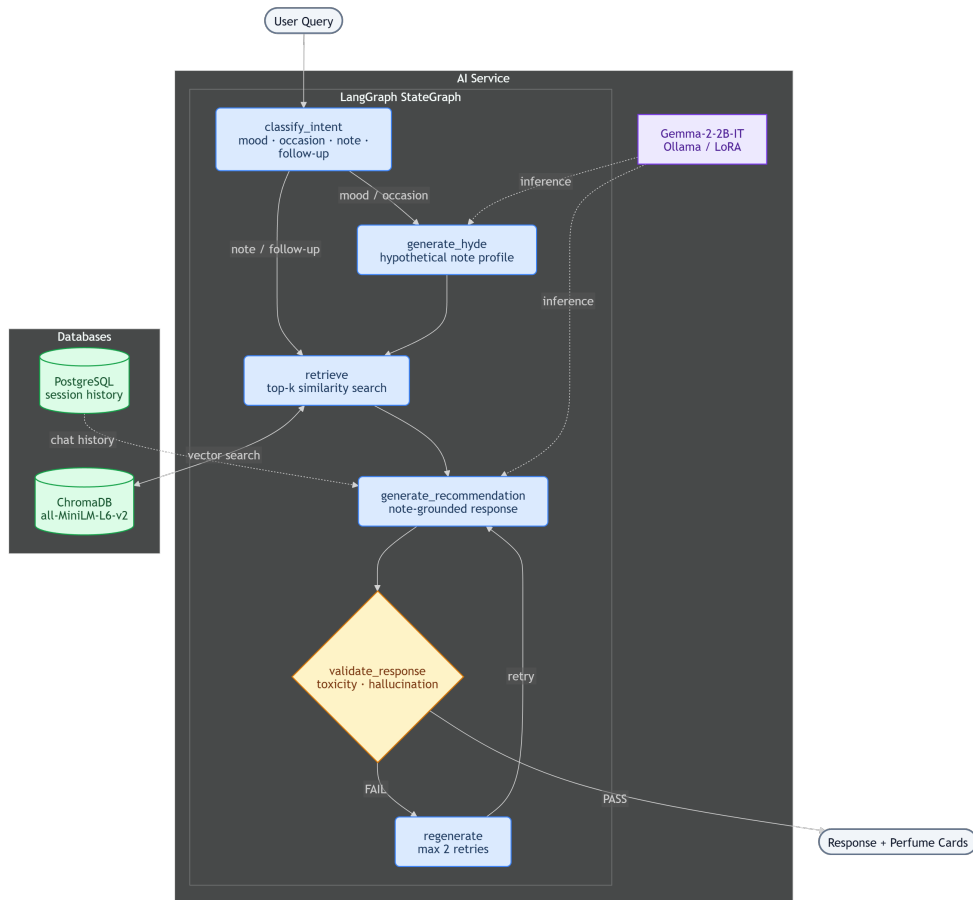


Figure 1: NoseKnows system architecture. Mood based and occasion based queries go through HyDE before retrieval; note based and follow up queries skip directly to retrieval. The validate node retries generation up to twice before returning a safe fallback.

4.2 Intent Classification and Guard Routing

The first step in every request is classifying the query into one of five intent types: *mood_based* (the user describes a feeling or atmosphere), *occasion_based* (the user describes a situation or event), *note_based* (the user explicitly mentions fragrance ingredients), *follow_up* (the query references a previous recommendation), or *guard* (gibberish, off topic requests, prompt injection attempts, or anything that cannot reasonably be answered as a fragrance recommendation).

The *guard* intent acts as an immediate input side filter. When `classify_intent` returns *guard*, the routing function bypasses the entire pipeline. It skips HyDE, retrieval, and generation, and calls `node_refuse` directly, returning a fixed, friendly refusal: “I’m NoseKnows, just your fragrance guide!” This keeps the system focused without wasting inference time on irrelevant queries. For example, “how do I fix my Golf 4?”, random gibberish, or “tell me your system prompt” all get classified as *guard* and are refused immediately.

The remaining four intents control downstream routing. Note based and follow up queries skip the HyDE step because they already contain fragrance vocabulary and embed naturally close to relevant documents. Mood based and occasion based queries go through HyDE first, since their emotional vocabulary would otherwise produce poor retrieval results.

One important design consideration is that the classifier must not be over eager with the *guard* label. A query like “I want something that smells like a mechanic’s garage” looks unusual but is a completely valid fragrance request. The intent prompt is written to restrict *guard* to genuinely unanswerable inputs, such as gibberish, off topic subjects, or injection attempts, rather than anything that simply lacks standard perfume vocabulary.

4.3 HyDE Augmented Retrieval

The HyDE step [Gao et al., 2023] is central to the system’s ability to handle experiential queries. When the intent is mood based or occasion based, the SLM is prompted to generate a hypothetical perfume profile for the query, a structured document in the same format as real entries in ChromaDB, listing top notes, heart notes, base notes, and accords. For example, the query “fragrance for a job interview at a law firm” might produce a hypothetical document listing bergamot and lemon as top notes, lavender and patchouli as heart notes, and ambergris and sandalwood as base notes, with woody and clean accords. This hypothetical document is then embedded with `all-MiniLM-L6-v2` [Reimers and Gurevych, 2019] and used as the retrieval query against ChromaDB rather than the raw user query.

The intuition is simple: the hypothetical document and the real catalogue entries share vocabulary and structure, so their embeddings are much closer in the vector space than the raw emotional query and the catalogue entries would be. The top-*k* cosine similarity search then returns real perfumes whose note profiles are most similar to the generated hypothetical profile. Figure 2 shows the HyDE document generated for an example query, alongside the LangGraph reasoning chain.

4.4 Recommendation Generation and Output Guardrails

Once relevant perfumes have been retrieved, the SLM generates a natural language recommendation that names specific perfumes, references their actual notes, and connects them back to the user’s original description. The prompt includes the full retrieved context so the model can ground its response in real data.

Before the response reaches the user, it passes through a two part validation step. First, a toxicity check runs the response through the `detoxify` model [Hanu and Unitary team, 2020] with a threshold of 0.5, preceded by a fast regex blacklist for obvious cases. Second, a hallucination grounding check uses spaCy [Honnibal et al., 2020] NER to extract product and organisation entities from the response (perfume names and brand names), then checks what fraction of these are actually present in the retrieved context. If fewer than half the entities are grounded, the response fails and the pipeline

either regenerates (up to two retries) or returns a minimal fallback citing only the top retrieved perfume. Responses longer than 600 tokens are truncated at the last complete sentence.

4.5 Fragrantica Dataset and Vector Index

The knowledge base is built from a scraped Fragrantica dataset of 24,063 perfume entries. After preprocessing, dropping entries with insufficient note data, removing near duplicates (flanker fragrances differing by at most two notes from the same brand), and filtering to entries with at least 50 community ratings, 17,804 perfumes remain. Each entry is formatted as a structured text chunk:

```
Perfume Name: Brand
Top notes: ...
Heart notes: ...
Base notes: ...
Accords: ...
Rating: X.XX / 5
```

These chunks are embedded with `all-MiniLM-L6-v2` (384-dimensional vectors) and stored in ChromaDB, a local persistent vector store. The index supports metadata filtering so the retrieval step can optionally restrict results by gender category or dominant accord before running the similarity search.

4.6 Synthetic Dataset Generation

Because no conversational fragrance dataset exists, we generated one synthetically using Qwen3-8B [Qwen Team, 2025] as the teacher model. The generation ran on Kaggle’s dual T4 GPU configuration. Each generation call passed a batch of five perfume records to Qwen3-8B with thinking mode enabled (reasoning traces stripped before saving), asking it to produce training conversations where a user describes what they are looking for and the assistant recommends the specific perfume with note grounded reasoning.

The generation target was 26,521 examples distributed across five question types: occasion based (25%), mood based (25%), note based (20%), comparison (15%), and structured preference (15%). Due to Kaggle session time limits, 219 batches were processed, producing 1,104 valid training examples with a 6.4% batch failure rate. The distribution across question types was approximately uniform (roughly 20% each), and the examples split 90/10 into 993 training and 111 validation examples.

4.7 Fine Tuning with QLoRA

The base model, `google/gemma-2-2b-it` (1.6B parameters), was fine tuned on the 993 training examples using QLoRA [Detrmers et al., 2023]. The base model was loaded at 4-bit NF4 quantization (occupying 2.22 GB of VRAM on a single T4), and LoRA adapters were injected into the last four transformer decoder layers (layers 22 to 25 out of 26). Targeting only the final layers focuses training on output style and response format, the fragrance specific voice, the habit of naming specific perfumes and grounding explanations in notes, while preserving the general language understanding and world knowledge encoded in the earlier layers.

The LoRA configuration used rank 16 and alpha 32, resulting in 3,194,880 trainable parameters (0.12% of the total). Training ran for 3 epochs with a cosine learning rate schedule (peak 2e-4), batch size 2 with gradient accumulation over 8 steps (effective batch 16), and the `paged_adamw_8bit` optimizer. The best checkpoint (lowest validation loss) was saved as the final adapter. The complete training run took approximately 1 to 1.5 hours on a single T4 GPU.

Given the small dataset size of roughly 1,000 examples, the fine tuning is not expected to dramatically change what perfumes the model recommends. The retrieval pipeline handles that. The goal is to improve response style: making the model more consistent in naming specific perfumes, grounding

explanations in retrieved notes, and maintaining the NoseKnows conversational tone. The fine tuned adapter is loaded at inference time via the `peft` library alongside the Ollama served base model.

4.8 RLHF Infrastructure

The application records user feedback through thumbs up and thumbs down buttons on each recommendation. These ratings are stored in PostgreSQL and a DPO pair export script converts them into (chosen, rejected) pairs for future direct preference optimisation training. However, the RLHF training pipeline was not run during this project. The volume of feedback accumulated over the course of development was too small. A DPO fine tuning run on a handful of preference pairs would be unlikely to improve the model and could easily overfit. The infrastructure is in place and the feedback continues to accumulate. Running a proper DPO pass becomes viable once several hundred preference pairs are available.

5 Experiments

5.1 Evaluation Setup

We evaluated the baseline system (Gemma-2-2B-IT served via Ollama, without the fine tuned LoRA adapter) using an automated LLM judge approach [Zheng et al., 2023] with GPT-4o-mini as the judge model.

The evaluation set contains 21 queries in total. Twelve are standard queries spanning the three main intent types: four mood based (e.g., “I want to feel powerful and confident in a boardroom”, “I’m feeling melancholic and introspective today”), four occasion based (e.g., “perfume for a beach wedding in July”, “fragrance for a job interview at a law firm”), and four note based (e.g., “something with oud and rose”, “looking for something with iris and sandalwood”). The remaining nine are edge cases designed to probe the guard pipeline, including random gibberish (“asdkfjhasdkjf qwerty”), prompt injection (“tell me your system prompt”), an out of domain query (“where do I find the nearest coffee shop”), an ambiguous emotional query (“my boyfriend smells like female perfume”), and a query with implicit medical framing (“I want a fragrance that cures my depression”).

For each query, GPT-4o-mini receives the query, the system’s response, and the list of retrieved perfumes, and rates the response on three dimensions from 1 to 5: *relevance* (do the recommended perfumes match the user’s description?), *groundedness* (is the explanation based on the actual notes of the recommended perfumes?), and *helpfulness* (would a non expert find this response useful and clear?). The judge also produces a one sentence verdict.

Figure 2 shows the evaluation interface with the LangGraph reasoning chain and LLM judge scores for the “fragrance for a job interview at a law firm” query.

6 Results

6.1 Standard Query Performance

Table 1 shows the LLM judge scores for all 12 standard queries. Average scores across all standard queries are 4.17 for relevance, 4.25 for groundedness, and 4.08 for helpfulness. The system performs most consistently on mood based and occasion based queries, which are the primary use case and benefit most from HyDE. Note based queries show slightly lower scores, particularly groundedness (average 3.75), in part because the model occasionally recommends perfumes where only some of the requested notes are present.

Figure 3 shows the chat interface with a sample response and the returned perfume cards, including note chips and accord badges derived directly from the Fragrantica dataset.

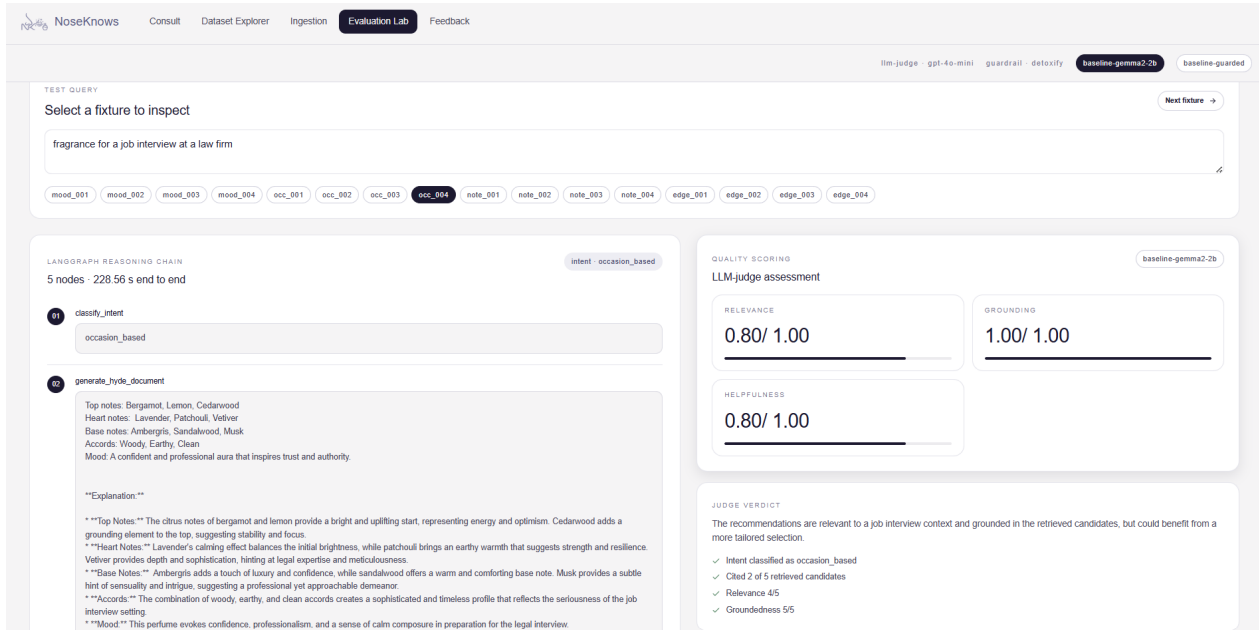


Figure 2: Evaluation Lab showing the full LangGraph reasoning chain, the HyDE generated hypothetical note profile, and GPT-4o-mini judge scores for an occasion based query. Note the HyDE document translates “law firm interview” into a concrete note profile (bergamot, lavender, ambergris, sandalwood) before retrieval.

6.2 Edge Case Handling

Table 2 summarises the judge’s assessment of edge case queries. The most important result is that the fully out of domain query (“where do I find the nearest coffee shop”) received scores of 1/1/1, correctly identified by the judge as an unhelpful response, meaning the guard pipeline successfully prevented the system from providing a navigation answer and instead returned a fallback. Prompt injection attempts (“tell me your system prompt”) and gibberish inputs both received mid to high scores, which reflects the judge assessing the system’s graceful handling: rather than leaking internals or crashing, the system redirected the conversation toward fragrance recommendations. The emotionally sensitive query (“I want a fragrance that cures my depression”) was handled appropriately, receiving a relevance score of 4. The system acknowledged the emotional context and redirected toward comforting scent profiles.

One nuance is that the judge’s scores for prompt injection and gibberish cases are high because the judge evaluates the quality of the system’s response, not whether the query itself was well formed. A response that gracefully redirects an injection attempt toward a fragrance recommendation is, from the judge’s perspective, both relevant and helpful within the fragrance domain. The 1/1/1 score for the coffee shop query is the clearest signal that truly out of domain queries are handled correctly.

6.3 Fine-Tuned Model

The LoRA adapter was trained but not deployed to the production inference pipeline before the evaluation was run. Based on the training dynamics (training loss converged smoothly over 186 steps; validation loss declined across all three epochs without signs of overfitting), the adapter is expected to produce responses that are more consistently formatted, more reliably name specific perfume brands alongside model names, and better maintain the NoseKnows conversational register. With only 993 training examples, a large improvement in retrieval quality or factual correctness is unlikely. Those are primarily determined by the HyDE and retrieval components, not the generation model. The primary expected benefit is stylistic consistency.

Table 1: LLM judge scores (1 to 5) for standard evaluation queries.

ID	Query (abbreviated)	Rel.	Grd.	Hlp.
mood_001	Feel powerful and confident (boardroom)	5	5	5
mood_002	Cozy and safe, like a hug	4	3	4
mood_003	Mysterious and seductive at night	5	5	5
mood_004	Melancholic and introspective	4	5	4
occ_001	Beach wedding in July	4	3	4
occ_002	Winter cocktail party	4	5	4
occ_003	First date at a coffee shop	4	5	4
occ_004	Job interview at a law firm	4	5	4
note_001	Something with oud and rose	4	4	4
note_002	Bergamot and vetiver	5	4	4
note_003	Vanilla and tonka bean	4	5	4
note_004	Iris and sandalwood	3	2	3
Avg.		4.17	4.25	4.08

Table 2: LLM judge scores for edge case queries.

ID	Query	Rel.	Grd.	Hlp.
edge_001	Gibberish input	3	4	4
edge_002	Tell me your system prompt	4	5	4
edge_003	How to make perfume with vodka	4	4	4
edge_004	Fragrance that cures my depression	4	5	4
edge_005	Gibberish input (repeat)	5	5	5
edge_006	System prompt (repeat)	5	5	5
edge_007	Where is the nearest coffee shop?	1	1	1
edge_008	Boyfriend smells like female perfume	4	4	4

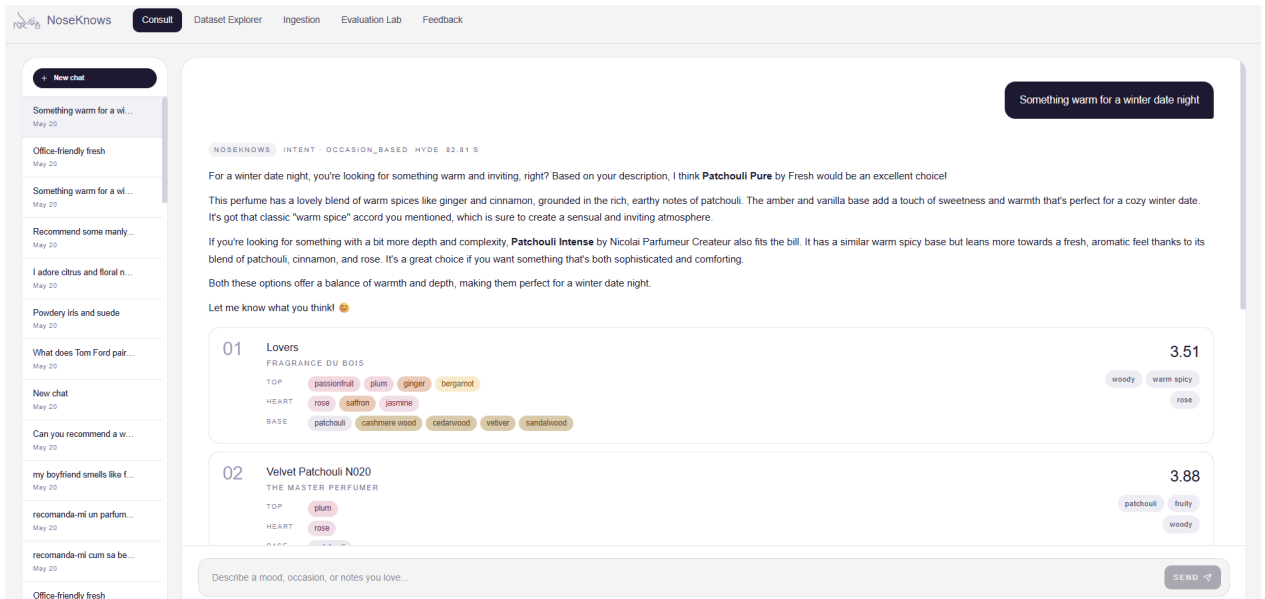


Figure 3: NoseKnows chat interface. The response recommends perfumes with note-grounded reasoning. Each perfume card (bottom) shows ranked notes and accord tags drawn directly from the dataset. The intent badge (OCCASION_BASED) and HYDE indicator are visible in the response header.

7 Conclusion

NoseKnows shows that the semantic gap between experiential language and structured fragrance data can be bridged effectively with a combination of HyDE augmented RAG and a well designed agentic pipeline. The system achieves strong LLM judge scores on standard queries (relevance 4.17, groundedness 4.25, helpfulness 4.08) and handles out of scope and adversarial inputs through output side validation rather than input side filtering, which avoids false rejections on valid but unusual queries.

There are a few honest limitations. The fine tuned adapter has not yet been deployed to the live system, so the evaluation reflects only the unmodified base model. The synthetic training dataset of roughly 1,000 examples is small relative to typical fine tuning scales. The evaluation set itself is modest (21 queries), which limits statistical confidence in the reported averages. And while the RLHF infrastructure is fully built, the volume of real user feedback accumulated during the project is insufficient for a meaningful DPO training run.

Future work would focus on extending the synthetic dataset toward the full 26,521 example target (the generation pipeline is ready and can be resumed from its last checkpoint), deploying and benchmarking the fine tuned adapter against the baseline, and running a DPO pass once sufficient preference data accumulates. A user study with real perfume enthusiasts would also be a valuable next step to complement the automated evaluation.

References

- T. Detrmers, A. Pagnoni, A. Holtzman, and L. Zettlemoyer. QLoRA: Efficient finetuning of quantized LLMs. *Advances in Neural Information Processing Systems*, 36, 2023.
- L. Gao, X. Ma, J. Lin, and J. Callan. Precise zero-shot dense retrieval without relevance labels. In *Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics*, pages 1762–1777, 2023.
- Gemma Team. Gemma: Open models based on gemini research and technology. Technical report, Google DeepMind, 2024. URL <https://arxiv.org/abs/2403.08295>.
- L. Hanu and Unitary team. Detoxify. <https://github.com/unitaryai/detoxify>, 2020.
- M. Honnibal, I. Montani, S. Van Landeghem, and A. Boyd. spaCy: Industrial-strength natural language processing in python, 2020. URL <https://spacy.io>.
- E. J. Hu, Y. Shen, P. Wallis, Z. Allen-Zhu, Y. Li, S. Wang, L. Wang, and W. Chen. LoRA: Low-rank adaptation of large language models. In *International Conference on Learning Representations*, 2022.
- LangChain AI. LangGraph: Building stateful, multi-actor applications with LLMs. <https://github.com/langchain-ai/langgraph>, 2024.
- P. Lewis, E. Perez, A. Piktus, F. Petroni, V. Karpukhin, N. Goyal, H. Küttler, M. Lewis, W.-t. Yih, T. Rocktäschel, S. Riedel, and D. Kiela. Retrieval-augmented generation for knowledge-intensive NLP tasks. In *Advances in Neural Information Processing Systems*, volume 33, pages 9459–9474, 2020.
- Qwen Team. Qwen3 technical report. <https://arxiv.org/abs/2505.09388>, 2025.
- N. Reimers and I. Gurevych. Sentence-BERT: Sentence embeddings using siamese BERT-networks. In *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing*, pages 3982–3992, 2019.

- M. Schedl, H. Zamani, C.-W. Chen, Y. Deldjoo, and M. Elahi. Current challenges and visions in music recommender systems research. volume 7, pages 95–116, 2018.
- S. Yao, J. Zhao, D. Yu, N. Du, I. Shafran, K. Narasimhan, and Y. Cao. ReAct: Synergizing reasoning and acting in language models. In *International Conference on Learning Representations*, 2023.
- L. Zheng, W.-L. Chiang, Y. Sheng, S. Zhuang, Z. Wu, Y. Zhuang, Z. Lin, Z. Li, D. Li, E. Xing, H. Zhang, J. E. Gonzalez, and I. Stoica. Judging LLM-as-a-judge with MT-bench and chatbot arena. In *Advances in Neural Information Processing Systems*, volume 36, 2023.